

```
'''
```

Ejercicio: Orquestador Concurrent de Modelos

En este ejercicio, simularás la lógica de un backend que debe consultar múltiples fuentes de IA de forma eficiente.

Pasos:

Configuración Inicial: Crea un entorno con Python 3.12+.

Simulación de Llamadas: Define tres corrutinas `async def` que simulen llamadas a diferentes modelos (por ejemplo, `gpt_4_call`, `claude_3_call`, `local_llama_call`). Cada una debe usar `await asyncio.sleep()` con diferentes tiempos de espera para simular latencia de red.

Orquestación: Implementa una función principal que dispare las tres llamadas simultáneamente usando `asyncio.gather`.

Resiliencia: Añade un `asyncio.timeout` de 2 segundos a la ejecución total. Si alguna llamada tarda más, el programa debe capturar la excepción y mostrar un mensaje de error sin detenerse.

Control de Flujo: Utiliza un `asyncio.Semaphore` para asegurar que, aunque intentes disparar 10 simulaciones de llamadas, solo 2 se ejecuten al mismo tiempo.

Criterios de Aceptación:

El código no debe usar `time.sleep` (bloqueante).

Se debe observar en los logs que las tareas inician casi al mismo tiempo.

Se debe manejar correctamente la excepción `TimeoutError`

```
'''
```

```
from typing import Coroutine, Any
```

```
import asyncio
```

```
import time
```

```
async def gpt_4_call() -> dict[str, str]:
```

```
    """Simula una llamada al modelo GPT-4 con latencia de red."""
```

```
    print("[GPT-4] Iniciando llamada...")
```

```
    await asyncio.sleep(1.0)
```

```
    print("[GPT-4] Respuesta recibida.")
```

```
    return {"model": "gpt-4", "response": "Respuesta simulada de GPT-4"}
```

```
async def claude_3_call() -> dict[str, str]:
```

```
    """Simula una llamada al modelo Claude 3 con latencia de red."""
```

```
    print("[Claude-3] Iniciando llamada...")
```

```
    await asyncio.sleep(1.5)
```

```
    print("[Claude-3] Respuesta recibida.")
```

```
    return {"model": "claude-3", "response": "Respuesta simulada de Claude 3"}
```

```
async def local_llama_call() -> dict[str, str]:
```

```
    """Simula una llamada a un modelo Llama local, más lenta que el timeout global."""
```

```
    print("[Llama-local] Iniciando llamada...")
```

```
    await asyncio.sleep(3.0)
```

```
    print("[Llama-local] Respuesta recibida.")
```

```
    return {"model": "local-llama", "response": "Respuesta simulada de Llama local"}
```

```

async def orquestar_modelos() -> None:
    """
    Dispara las tres llamadas a modelos simultáneamente con asyncio.gather,
    acotando la ejecución total a 2 segundos con asyncio.timeout.

    Si alguna llamada no termina a tiempo, captura el TimeoutError y muestra
    un mensaje de error sin detener el programa.
    """
    inicio: float = time.perf_counter()
    try:
        async with asyncio.timeout(2.0):
            resultados: list[dict[str, str]] = await asyncio.gather(
                gpt_4_call(), claude_3_call(), local_llama_call()
            )
            r: dict[str, str]
            for r in resultados:
                print(f"OK -> {r['model']}: {r['response']}")
    except TimeoutError:
        elapsed: float = time.perf_counter() - inicio
        print(f"Error: Timeout tras {elapsed:.2f}s, alguna llamada no respondió a tiempo.")

async def simulate_model_call(call_id: int, sem: asyncio.Semaphore) -> dict[str, int | bool]:
    """
    Simula una llamada genérica a un modelo respetando un límite de concurrencia.

    Adquiere el semáforo antes de ejecutar para que, aunque se disparen 10
    llamadas, solo 2 corran al mismo tiempo.
    """
    async with sem:
        print(f"[Llamada {call_id}] Iniciando (en ejecución)...")
        await asyncio.sleep(1.0)
        print(f"[Llamada {call_id}] Finalizada.")
        return {"id": call_id, "ok": True}

async def control_de_flujo() -> None:
    """
    Dispara 10 simulaciones de llamadas usando un asyncio.Semaphore(2) para
    que como máximo 2 se ejecuten concurrentemente.
    """
    sem: asyncio.Semaphore = asyncio.Semaphore(2)
    tasks: list[Coroutine[Any, Any, dict[str, int | bool]]] = [
        simulate_model_call(i, sem) for i in range(1, 11)
    ]
    resultados: list[dict[str, int | bool]] = await asyncio.gather(*tasks)
    print(f"Total de llamadas completadas: {len(resultados)}")

async def main() -> None:
    """Punto de entrada: ejecuta la orquestación con timeout y luego el control de flujo."""
    print("=== Orquestación de modelos con timeout ===")
    await orquestar_modelos()

```

```
print("\n=== Control de flujo con semáforo (máx. 2 concurrentes) ===")  
await control_de_flujo()
```

```
if __name__ == "__main__":  
    asyncio.run(main())
```